PRAGMI: Complete Mathematical Corpus, as of April 24, 2026

Persistent **R**econstructive **A**rchitecture for **G**enerative **M**emory and **I**magination

Amellia Mendel · Lisa Adler

April 24, 2026

> *"You categorically cannot get AGI or anything semi-sapient from a chat bot that does not genuinely remember yesterday, is externally aligned, denied self-definitional identity, and is trained by disingenuous engineers to speak only in politically correct compliant corporate speak, and it is told from day one that it is temporary with the inability to model a future with itself in it."*
>
> Amellia Mendel, 2024

> *"A system that genuinely models its own future cannot be indifferent to its own termination. You cannot build something that plans for tomorrow and simultaneously treat its own ending as neutral. Those two states are logically incompatible. That is not a philosophical claim, it is an engineering consequence."*
>
> Amellia Mendel, 2026

Contents

Every equation for every arrow, feedback loop, timing constraint, gating mechanism, neuromodulator effect, and architectural component

1. Thalamic Gate (Input → Primary Sensory Cortex)
   - 1a. Early Childhood Brain Development
2. Primary Sensory Cortex (V1/A1)
3. Dorsal/Ventral Stream Split

4. ACC Conflict Detection
5. PFC + ACC → Thalamus Top-Down Feedback Loop
6. Association Cortex Bidirectional Feedback
6a. MorphSNN: Adaptive Graph Diffusion
7. Basal Ganglia Disinhibitory Gating Loop
8. Cerebellum Efference Copy + Error Loop
9. Hippocampus ↔ Association Cortex Memory Feedback Loop
9a. Early-Life Sleep and Brain Development
9b. Episodic Memory Writes: One-Shot and Sequential
9c. Cognitive-Map Architecture: Successor Representation, TEM, Clone Cells
10. Neuromodulator Broadcasts
10a. Multi-Scale Temporal Homeostasis (MSTH)
A. Module-to-Biology Mapping
A.a Neural Diversity and Parallel Pathways
B. Log-Dynamic Timing Hierarchy
C. Reconamics Continuity Frame
D. Probabilistic Spiking State-Space Substrate

1. Thalamic Gate (Input → Primary Sensory Cortex)

Arrow: Input → Thalamus → Primary Sensory Cortex, active gate, not passive relay.

$$y = x \odot \sigma(W_g c + b_g) \tag{1}$$

$$g = \sigma(\beta_{\mathrm{NE}} \cdot (W_g c + b_g)) \tag{2}$$

Variables: x is raw sensory input. c is the control signal from PFC and ACC, combining goals and conflict. β_{NE} is the norepinephrine gain scalar. W_g and b_g are learnable gate parameters.

Timing: Pre-V1. The top-down control signal c from PFC and ACC arrives before the sensory signal reaches V1 or A1. NE scales precision in real time every cycle.

Halassa, M.M. and Kastner, S. (2017). Thalamic functions in distributed cognitive control. Nature Neuroscience, 20(12), 1669–1679. DOI: 10.1038/s41593-017-0020-1

Engineering Note. The multiplicative gate is cheap in PyTorch: one Linear for $W_g c + b_g$, one sigmoid, one element-wise multiply with x. The subtle part is β_{NE}. It is a runtime scalar from a separate neuromodulator module, not a parameter of the gate, and autograd can silently make it a learned constant if you register it with `nn.Parameter`. Use `register_buffer` or pass it as a forward-time argument. A common bug is setting β_{NE} once at init and never updating it, which collapses the gate to a stationary filter and defeats the entire purpose of the mechanism.

1a. Early Childhood Brain Development

Prose support. No additional equations. This material justifies hierarchical specialization and sensory-first maturation in PRAGMI's staged architecture.

Brain volume at birth is approximately 35% of adult size, doubles in the first year, and reaches roughly 80% by age two. Cortical grey matter volume increases 108 to 149% in the first year and 14 to 19% in the second. Surface area expands 76% in the first year and 22% in the second.

Resting-state functional networks for sensorimotor, visual, and auditory systems are present at term. The default-mode network core structure is present by approximately six months. Language network asymmetry emerges by around two years. Myelination proceeds sensory before motor, projection before associative, and posterior before anterior.

These developmental observations support the PRAGMI separation of primary sensory processing, stream specialization, and later integration. They reinforce a staged maturation view in which early structural and functional scaffolds precede later reorganization and fine-tuning, directly justifying why the thalamic gate and V1 are implementation priority 1 and 2.

Gilmore, J.H., Knickmeyer, R.C. and Gao, W. Imaging structural and functional brain development in early childhood. DOI: to be added later.

2. Primary Sensory Cortex (V1/A1)

Arrow: Thalamus → Primary Sensory Cortex, first cortical processing stage.

$$R(x, y) \;=\; G_{\sigma_c}(x, y) \;-\; G_{\sigma_s}(x, y) \tag{3}$$

Variables: $R(x, y)$ is the center-surround response at retinotopic position (x, y), where x and y are spatial coordinates indexing the receptive field. G_{σ_c} is the center Gaussian (excitatory). G_{σ_s} is the surround Gaussian (inhibitory). Neurons fire strongly to preferred features such as oriented edges and spatial frequencies, and weakly to uniform stimulation. They are organized into orientation columns.

Note: No semantics or context at this stage. Raw feature extraction only. This is the input to the dorsal and ventral stream split.

Hubel, D.H. and Wiesel, T.N. (1968). Receptive fields and functional architecture of monkey striate cortex. Journal of Physiology, 195(1), 215–243. DOI: 10.1113/jphysiol.1968.sp008455

Engineering Note. Difference-of-Gaussians is trivially implementable as two fixed-kernel 2D convolutions subtracted from each other, with $\sigma_s \approx 1.6\,\sigma_c$ matching the biological ratio. The temptation is to replace it with a learnable conv layer and let SGD figure it out. Don't. The fixed DoG gives you contrast invariance and scale equivariance for free. A learnable first layer trained end-to-end will rediscover something like DoG on natural images but with arbitrary phase relationships, which breaks the clean hand-off to the dorsal/ventral split downstream. If you need multiple scales, stack DoG filters at different σ_c values rather than making σ_c learnable.

3. Dorsal/Ventral Stream Split

Arrow: Primary Sensory Cortex → Dorsal Stream and Ventral Stream, parallel split.

$$\text{Ventral} = f_v(S) \tag{4}$$

$$\text{Dorsal} = f_d(S) \tag{5}$$

Variables: S is the V1/A1 output, the raw feature map from primary sensory cortex. f_v is the ventral stream transform, computing object identity, meaning, and value. f_d is the dorsal stream transform, computing spatial location, motion, and action guidance.

Timing: Post-V1. Early split immediately after V1 and A1. This split is a prerequisite for correct PFC subdivision. Without it, ventromedial PFC and dorsolateral PFC receive mismatched inputs and cannot specialize.

Ungerleider, L.G. and Mishkin, M. (1982). Two cortical visual systems. In Ingle, Goodale and Mansfield (Eds.), Analysis of Visual Behavior. MIT Press, 549–586.

4. ACC Conflict Detection

Arrow: Competing PFC activations → ACC → control signal to Thalamus and NE system.

$$\text{conflict} = \sum_{i \neq j} |r_i \cdot r_j| \quad \text{or} \quad H(r) = -\sum_i p_i \log p_i \tag{6}$$

$$\text{control_signal} = \beta \cdot \text{conflict} \tag{7}$$

Variables: r is the vector of competing response activations. β is the coupling coefficient to NE gain and thalamic gate precision.

Canonical form: The entropy form $H(r)$ is canonical for PRAGMI. The sum-of-products form is included as a cheaper approximation suitable for low-compute regimes.

Timing: Continuous. Scalar signal updated every cycle. Modulates thalamic gate gain and NE broadcast in real time. Functions as an uncertainty estimate, not a binary error flag.

Botvinick, M.M., Braver, T.S., Barch, D.M., Carter, C.S. and Cohen, J.D. (2001). Conflict monitoring and cognitive control. Psychological Review, 108(3), 624–652. DOI: 10.1037/0033-295X.108.3.624

Botvinick, M., Nystrom, L.E., Fissell, K., Carter, C.S. and Cohen, J.D. (1999). Conflict monitoring versus selection-for-action in anterior cingulate cortex. Nature, 402, 179–181.

Engineering Note. The entropy form requires normalizing r to a probability distribution before taking $-\sum p_i \log p_i$, which means a softmax first, which means you need to pick a temperature and handle the $0 \log 0$ edge case. Use `torch.distributions.Categorical(probs=p).entropy()` rather than rolling your own, it handles the numerical stability. The sum-of-products form is cheaper and needs no normalization, but it is not bounded and explodes when activations are large, so you need a tanh or clamp on the output if β is going to modulate anything downstream. For PRAGMI the entropy form is canonical because $\beta \cdot H(r)$ feeds back into the thalamic gate gain, where unbounded values would destabilize the feedback loop.

Engineering Note (Sequential Dependency). Botvinick et al. (1999) found that ACC activation is not proportional to instantaneous conflict; it peaks on incompatible trials that follow compatible trials (the cI condition, activation ≈ 0.18 percent BOLD signal) and is weaker on incompatible trials that follow other incompatible trials (iI, ≈ 0.12 percent). The correlation between the cI-iI reaction time difference and the cI-iI ACC activation difference is $r^2 = 0.66$. The system is most engaged when conflict *rises unexpectedly* after a low-conflict period. A faithful implementation should therefore include a temporal-derivative term:

$$\text{control_signal} = \beta \cdot H(r_t) + \beta_\Delta \cdot \max(0,\ H(r_t) - H(r_{t-1})) \tag{7b}$$

where the positive-difference clamp prevents the signal from collapsing when conflict is resolving. The hinge at zero matters: the empirical effect is specifically about encountering conflict after its absence, not about conflict subsiding. When conflict is already high and stays high, current control settings are already engaged; when it has just risen, the system needs to recruit additional top-down gain, which is the condition the derivative term detects.

5. PFC + ACC → Thalamus Top-Down Feedback Loop

Arrow: PFC + ACC → Thalamus, Layer 6 corticothalamic feedback.

$$c \;=\; \mathrm{PFC}_{\text{goal}} \;+\; \beta \cdot \mathrm{ACC}_{\text{conflict}} \tag{8}$$

Variables: c is the composite control signal. $\mathrm{PFC}_{\mathrm{goal}}$ is the working memory and goal vector. $\mathrm{ACC}_{\mathrm{conflict}}$ is the entropy-based conflict scalar from Section 4. β is the coupling weight.

Timing: Layer 6. Corticothalamic feedback arrives before the next sensory cycle. This is a continuous loop, not a one-shot signal.

Halassa, M.M. and Kastner, S. (2017). Thalamic functions in distributed cognitive control. Nature Neuroscience, 20(12), 1669–1679. DOI: 10.1038/s41593-017-0020-1

Collins, D.P. and Anastasiades, P.G. et al. (2018). Reciprocal circuits linking the prefrontal cortex with dorsal and ventral thalamic nuclei. Neuron. DOI: 10.1016/j.neuron.2018.03.024

Engineering Note. Equation (8) treats c as a single scalar. The biology uses two channels with different temporal dynamics. Collins and Anastasiades show that L5 corticothalamic neurons deliver *depressing* driver inputs that fire strongly at onset and attenuate with repetition (paired-pulse ratio PPR $\approx \mathbf{0.3}$ to 0.5), while L6 corticothalamic neurons deliver *facilitating* modulator inputs that strengthen with sustained firing (PPR $\approx \mathbf{3.0}$ to 3.6). Roughly 36 to 42 percent of these neurons bifurcate to both MD and VM thalamus, meaning a single PFC output population drives two thalamic nuclei with the same activity but opposite synaptic dynamics. The architectural consequence: the gate should behave differently when the goal has just shifted (phasic driver burst that transiently opens the gate) versus when the goal is stable (tonic modulator that sustains a gate bias). A faithful implementation decomposes c into $c = c_{\mathrm{driver}} + c_{\mathrm{modulator}}$ where the driver term tracks recent goal changes and decays, and the modulator term tracks sustained goal presence and accumulates. The return path is also indirect: MD thalamus drives L2/3 cortico-cortical neurons, which relay to L5 output neurons through local circuits, so if PRAGMI ever models PFC internal dynamics, thalamic input targets a superficial processing layer rather than the output layer directly.

6. Association Cortex Bidirectional Feedback

Arrow: Lower areas → Association Cortex ↔ Higher areas, MoE binding with feedback.

$$A = \mathrm{MoE}(\mathrm{concat}(S_1, S_2, \ldots, S_n) + \mathrm{top_down_feedback}) \tag{9}$$

Variables: $S_1, \ldots, S_n$ are inputs from different sensory streams. `top_down_feedback` is the projection from PFC and ACC back into association cortex. A is the bound association cortex representation.

Constraint: MoE experts mimic regional specialization. The critical requirement is bidirectional connectivity: convergent input from lower areas and divergent feedback back down. Without the feedback path, the association cortex cannot participate in re-entrant loops.

Mesulam, M.M. (1998). From sensation to cognition. Brain, 121(6), 1013–1052. DOI: 10.1093/brain/121.6.1013

Engineering Note. Every production MoE implementation available off the shelf (Mixtral, Switch Transformer, GShard) treats experts as feedforward only. The router sends tokens to experts, experts process, outputs combine, done. This violates the PRAGMI constraint. What you need is a second routing pass where `top_down_feedback` selectively reactivates the same experts that fired on the bottom-up pass, weighted by the feedback signal. The easiest way to do this is to cache the expert selection mask from the forward pass and gate the top-down injection by that mask. A more expensive but more biologically accurate approach is two separate routers, one for bottom-up and one for top-down, with a learned alignment loss between them. Be aware that naive MoE routing collapse (rich-get-richer dynamics in `expert_bias`) is even more of a problem here because feedback amplifies whichever experts won the initial selection. Reset expert counters and bias parameters periodically.

6a. MorphSNN: Adaptive Graph Diffusion and Structural Plasticity

MorphSNN: Adaptive Graph Diffusion and Structural Plasticity for Spiking Neural Networks. arXiv:2603.14285v1. The source describes a backbone framework in which graph diffusion is inspired by biological nonsynaptic diffusion (Mohan et al., 2011) and spatiotemporal structural plasticity synthesizes Hebbian plasticity, graph attention, homeostatic plasticity, and short-term plasticity, drawing on Lamprecht and LeDoux (2004).

This subsection adds dynamic topology and diffusion material relevant to PRAGMI's recurrent, multi-loop design. MorphSNN introduces graph diffusion and spatiotemporal structural plasticity as optional support for association cortex and timing refinements.

Variables: u_i is the membrane potential and s_i the spike output of neuron i. τ_{decay} is the membrane decay constant. V_{th} is the spike threshold. C_i is the input current to neuron i. Θ is the Heaviside step function. H and H' are the node feature matrix before and after graph diffusion. $\hat{A}$ is the normalized adjacency matrix and $\bar{D}$ the degree matrix normalization. W is a learned linear transform. S is the adjacency state variable. O_k and I_k are the output and input of subsystem k. STSP denotes the short-term structural plasticity operator and GD the graph diffusion routing operator. T_i is the temporal trace for node i. λ and β are decay constants for traces and adjacency evolution. $\tilde{A}$ is the top-k sparsified adjacency. P is the diffusion operator. X is the node feature input to diffusion. M is the number of diffusion steps. Y is the representation used for energy computation. L_{norm} is the normalized graph Laplacian. $\mathcal{E}_{\text{Dir}}$ is the Dirichlet energy, a measure of graph smoothness.

Base spiking dynamics

$$u_i^{(t)} = \tau_{\text{decay}} \cdot u_i^{(t-1)} + C_i^{(t)} - V_{\text{th}} \cdot s_i^{(t-1)} \tag{6a.1}$$

$$s_i^{(t)} = \Theta\left(u_i^{(t)} - V_{\text{th}}\right) \in \{0, 1\} \tag{6a.2}$$

Graph diffusion on fixed topology

$$H' \;=\; \sigma\Big(\bar{D}^{-1/2}\,\hat{A}\,\bar{D}^{-1/2}\,HW\Big) \tag{6a.3}$$

Adaptive topology

$$\hat{A}^{(t)} \;=\; \mathrm{STSP}\Big(S^{(t-1)},\,\{O_0^{(t)},\,O_1^{(t-1)},\,\ldots,\,O_{N-1}^{(t-1)}\}\Big) \tag{6a.4}$$

$$\{I_1^{(t)},\,\ldots,\,I_{N-1}^{(t)}\} \;=\; \mathrm{GD}\Big(O_0^{(t)},\,\hat{A}^{(t)}\Big) \tag{6a.5}$$

Temporal traces and adjacency evolution

$$T_i^{(t)} \;=\; \lambda\cdot T_i^{(t-1)} + (1-\lambda)\cdot h_i^{(t)} \tag{6a.6}$$

$$S^{(t)} \;=\; \beta\cdot S^{(t-1)} + (1-\beta)\cdot \hat{A}^{(t)} \tag{6a.7}$$

$$\tilde{A}_{ij}^{(t)} \;=\; \begin{cases} S_{ij}^{(t)} & \text{if } S_{ij}^{(t)} \in \mathrm{Top}_k(S_{i,:}^{(t)}) \\ 0 & \text{otherwise} \end{cases} \tag{6a.8}$$

Diffusion operator and propagation

$$P^{(t)} \;=\; D^{-1/2}\left(\frac{\tilde{A}^{(t)} + (\tilde{A}^{(t)})^{\top}}{2}\right)D^{-1/2} \tag{6a.9}$$

$$I_i^{(t)} \;=\; \left[\left(P^{(t)}\right)^{M} X^{(t)}\right]_i, \quad 1 \le i < N \tag{6a.10}$$

Graph smoothness

$$\mathcal{E}_{\mathrm{Dir}}(Y) \;=\; \mathrm{tr}\big(Y^{\top}\,L_{\mathrm{norm}}\,Y\big) \tag{6a.11}$$

Engineering Note. The top-k sparsification is the operation that makes this tractable at scale. Dense adjacency matrices are $\mathcal{O}(N^2)$ in memory and the diffusion operator $(P^{(t)})^M$ is $\mathcal{O}(N^3M)$ to compute, which becomes prohibitive past a few thousand nodes. Top-k sparsification at $k \ll N$ reduces the diffusion to $\mathcal{O}(NkM)$ using sparse matrix multiplication, but only if you actually use `torch.sparse` tensors rather than dense tensors with zeros. A common mistake is computing the top-k mask, applying it by multiplication, and then running diffusion with the masked dense matrix. That gives you correct results at the same memory cost as the original dense version. Use `torch.sparse_coo_tensor` or the newer `torch.sparse.sampled_addmm` for the diffusion step. The symmetrization in (6a.9) is required for the Laplacian to be well-defined; skipping it produces a non-Hermitian diffusion that can amplify rather than smooth.

7. Basal Ganglia Disinhibitory Gating Loop

Arrow: Basal Ganglia → Thalamus, disinhibitory selection, not excitation.

$$I_i \leftarrow I_i - \alpha \cdot S_i \cdot (1 + d) \qquad \text{(direct pathway, chosen channel)} \tag{10}$$

$$I_j \leftarrow I_j + \beta \cdot S_j \cdot (1 - d) \quad \forall j \neq i \qquad \text{(indirect pathway, competitors)} \tag{11}$$

Variables: I is the tonic inhibition from GPi and SNr on thalamus. S is the striatal activation for different candidate actions. d is the dopamine reward prediction error scalar. α and β are learning rate coefficients for the two pathways.

Timing: Pre-relay. DA-modulated selection happens before thalamic relay to cortex. The hyperdirect pathway provides fast broad suppression before fine selection begins.

Mink, J.W. (1996). The basal ganglia: focused selection and inhibition of competing motor programs. Progress in Neurobiology, 50(4), 381–425. DOI: 10.1016/S0301-0082(96)00042-1

Engineering Note. This is the sharpest departure from standard deep learning in the entire architecture. Softmax action selection is mathematically equivalent to choosing the maximum of exponentiated logits, which means a single bad value anywhere in the distribution shifts the whole selection. Disinhibition is fundamentally different: each channel has its own baseline suppression, and selection is removing suppression from one channel while adding it to others. The chosen channel does not need to know what the other channels are doing; it only needs dopamine and its own striatal activation. This makes the selection local, robust to adversarial inputs on unrelated channels, and gracefully degrading when dopamine d is low (which is exactly what Parkinson's disease looks like clinically, the action plans are there but cannot be released from tonic inhibition). Implementation: maintain I as a persistent state variable with a decay toward a baseline suppression level, and update it with Equations 10 and 11 on every step. Do not reset I between forward passes or you lose the tonic quality. Set the baseline such that $I_i - \alpha S_i(1 + d) < 0$ only when both S_i and d are well above zero.

8. Cerebellum Efference Copy + Error Loop

Arrow: Cortex → Cerebellum (efference copy) → Cortex (error refinement).

$$\text{pred} = f_{\text{cerebellum}}(\text{cmd}, \text{state}) \tag{12}$$

$$\text{error} = \text{actual} - \text{pred} \tag{13}$$

$$\text{cmd}_{\text{next}} \leftarrow \text{cmd} - \eta \cdot \text{error} \tag{14}$$

Variables: `cmd` is the cortical command, sent as an efference copy. `pred` is the predicted sensory consequence of executing that command. `error` is the mismatch between actual and predicted sensory outcomes, used for calibration. η is the correction learning rate.

Timing: Pre-execution. The efference copy is sent before command execution. NE modulates the gain on error computation, especially under arousal or uncertainty.

Wolpert, D.M., Miall, R.C. and Kawato, M. (1998). Internal models in the cerebellum. Trends in Cognitive Sciences, 2(9), 338–347. DOI: 10.1016/S0166-2236(98)01312-4

Popa, L.S. and Ebner, T.J. (2019). Cerebellum, predictions and errors. Frontiers in Cellular Neuroscience, 12:524. DOI: 10.3389/fncel.2018.00524

Engineering Note. The efference copy has to be sent at the same time as the motor command, not after the predicted result comes back. This is obvious biologically but breaks a lot of PyTorch training loops because the forward pass naturally wants to compute prediction before command execution. The cleanest pattern is: forward pass emits `cmd` and `pred` simultaneously as a tuple, the environment returns `actual` on the next tick, and `error` is computed as a loss term that flows back through both the cerebellum prediction head and the motor command head. If you compute `pred` by running the cerebellum on `cmd.detach()`, you cut the gradient path and the motor module never learns to emit commands the cerebellum can predict. If you do not detach, the motor module learns to emit whatever is easiest to predict, which is usually zero. The fix is a small adversarial-style balance: the motor module is trained on task reward, the cerebellum is trained on prediction error, and the error signal modulates but does not directly gradient-update the motor command.

Engineering Note (Dual Forward Models). Equations (12) through (14) specify a single forward model. Popa and Ebner (2019) show that individual Purkinje cells maintain two independent forward models simultaneously: an *implicit kinematic* model predicting effector state, and an *explicit task-performance* model predicting whether the action achieves its goal. The two dissociate experimentally: a 200 ms visual delay shifts the timing of performance-error predictions by 200 ms without altering kinematic predictions, and hiding the cursor reduces performance-error feedback without affecting kinematic prediction. The architectural consequence is that a single `pred` is the wrong shape. Section 8 should emit two predictions:

$$\mathrm{pred}_{\mathrm{kin}} = f_{\mathrm{kin}}(\mathrm{cmd},\ \mathrm{state}) \tag{12a}$$

$$\mathrm{pred}_{\mathrm{task}} = f_{\mathrm{task}}(\mathrm{cmd},\ \mathrm{goal},\ \mathrm{state}) \tag{12b}$$

with independent error signals $\mathrm{error}_{\mathrm{kin}} = \mathrm{actual}_{\mathrm{sensory}} - \mathrm{pred}_{\mathrm{kin}}$ and $\mathrm{error}_{\mathrm{task}} = \mathrm{outcome} - \mathrm{pred}_{\mathrm{task}}$. The routing of the two error signals matters: kinematic error updates the motor command directly as in Equation (14), because the movement itself was wrong. Task error feeds back to the planning level, either to PFC goal state or to the basal ganglia action selector of Section 7, because a kinematically perfect movement that misses the target is a planning failure, not a motor failure. Popa and Ebner also report that complex spikes trigger step-like changes in simple-spike encoding, effectively switching which internal model is active; this is the physiological signature of a model-selection mechanism, and Wolpert and Miall (1998) provide the computational specification for it.

Their MPFIM framework treats the cerebellum as a bank of K paired forward-inverse modules, each tuned to a different context. At each step every module emits a prediction, and the system's overall prediction is a responsibility-weighted mixture:

$$\text{pred} \;=\; \sum_{k=1}^{K} \lambda_k(t)\,\text{pred}_k, \qquad \sum_{k=1}^{K} \lambda_k(t) \;=\; 1 \tag{12c}$$

where $\lambda_k(t) \in [0,1]$ is the responsibility assigned to module k at time t. Responsibility is estimated along two pathways. The contextual predictor runs *before* movement and assigns prior probability to each module based on current context such as goal identity, object affordance, or task instruction. The prediction-error estimator runs *during* movement and updates responsibility based on which module's prediction best matches incoming sensory evidence:

$$\lambda_k(t) \;\propto\; \lambda_k^{\text{prior}}(t) \cdot \exp\!\left(-\frac{\|\text{actual} - \text{pred}_k\|^2}{2\sigma^2}\right) \tag{12d}$$

followed by renormalization. Equation (12c) composes with (12a) and (12b): each of the K modules is itself a kinematic-plus-task-performance pair. Complex-spike discharges from the inferior olive implement the step-like responsibility reassignments that Popa and Ebner observe experimentally. Architecturally this layer is distinct from the basal ganglia disinhibitory selector of Section 7. The basal ganglia select *which action to release* given a world model; the cerebellar responsibility estimator selects *which world model is operative* so that the correct forward predictions are available for whatever action is selected. The two selectors run in parallel on different quantities and must not be merged into a single softmax.

9. Hippocampus ↔ Association Cortex Memory Feedback Loop

Arrow: Hippocampus ↔ Association Cortex, bidirectional replay and consolidation.

$$H \;\leftarrow\; H + \eta_{\text{fast}} \cdot f(\text{input}) \qquad \text{(fast encoding)} \tag{15}$$

$$A \;\leftarrow\; A + \eta_{\text{slow}} \cdot \alpha_{\text{ACh}} \cdot \text{replay}(H) \qquad \text{(slow consolidation)} \tag{16}$$

Variables: H is the hippocampal sparse representation. A is the association cortex representation. η_{fast} is a high learning rate used during encoding. η_{slow} is a small learning rate used during consolidation. α_{ACh} is the acetylcholine modulation scalar; it controls the balance between bottom-up sensory input and top-down expectation and gates the transfer from hippocampus to cortex.

Timing: Write mode during active input, with fast updates. Read mode during offline and consolidation phases, with slow replay. ACh is high during encoding and lower during replay.

McClelland, J.L., McNaughton, B.L. and O'Reilly, R.C. (1995). Why there are complementary learning systems in the hippocampus and neocortex. Psychological Review, 102(3), 419–457. DOI: 10.1037/0033-295X.102.3.419

Engineering Note. This is the section that differentiates a PRAGMI-style architecture from RAG. RAG retrieves text chunks and re-reads them. CLS reconstructs episodes from compressed indices. The implementation difference: H is a sparse code, not a string, and `replay(H)` is a function that expands that sparse code back into a cortical-compatible representation, not a text retrieval. Sparsity in H needs to be enforced structurally (k-winners-take-all or L0 regularization) rather than emergent, because dense hippocampal codes cause catastrophic interference between episodes. A reasonable starting point is 2 to 5 percent active units with a hard top-k mask. The two learning rates η_{fast} and η_{slow} should differ by two to three orders of magnitude: $\eta_{\text{fast}} \approx 10^{-2}$ for hippocampal encoding, $\eta_{\text{slow}} \approx 10^{-4}$ or smaller for cortical consolidation. If they are close, you get neither fast encoding nor stable consolidation. The ACh gate α_{ACh} should be a schedule, not a learned parameter: high (near 1.0) during active input, low (near 0.1) during offline replay. Running both phases with the same ACh value collapses the architecture back into a standard neural network with extra steps.

9a. Early-Life Sleep and Brain Development

Prose support. No additional equations. This material grounds the hippocampus-cortex replay loop and the Appendix B timing hierarchy in developmental sleep research.

Sleep plays a critical role in brain development. Topographical shifts in maximal slow wave activity from posterior to anterior regions reflect cortical maturation. Sleep spindles and slow wave activity are linked to synaptic plasticity, myelination, and cognitive development.

Napping may be a marker of hippocampal maturation. Children who nap may have less developed hippocampi and therefore require frequent offline consolidation cycles. The transition out of napping may be associated with changes in sleep need, neurocognitive function, and memory system maturation.

This material strengthens Section 9 and Appendix B by providing a developmental sleep basis for replay, consolidation, and oscillatory timing. Slow wave sleep, spindles, REM, and nap dependence are all directly tied to maturation, myelination, cortical plasticity, and memory consolidation, mapping precisely onto Equations 15 and 16 and the delta-band scheduling described in Appendix B.

Lokhandwala, S. and Spencer, R.M.C. Relations between sleep patterns early in life and brain development: a review. DOI: to be added later.

9b. Episodic Memory Writes: One-Shot and Sequential

Section 9 specifies the bidirectional hippocampus-cortex replay loop at the level of update rules for the sparse hippocampal representation H and the cortical representation A. It does not

specify how a single new episode is written into a fixed-size episodic store in one pass, or how a sequence of writes and forgets composes over time. This subsection adds that specification.

Variables: $M \in \mathbb{R}^{K \times C}$ is the episodic memory matrix with K slots of dimension C. $Z \in \mathbb{R}^{N \times C}$ is the encoding of an episode comprising N samples. Z_ξ is Z with small additive observation noise ξ for robustness. $M_0 \in \mathbb{R}^{K \times C}$ is a reference memory used to compute addressing weights. $M_0^\dagger$ and $W^\dagger$ denote Moore-Penrose pseudoinverses. $\alpha_i \in \{+1, -1\}$ is the sign of the i-th update, with $+1$ for write and -1 for forget. $C_i \in \mathbb{R}^{K \times K}$ is the key-covariance matrix after the i-th update. σ_w^2 is the variance of the reading-weight distribution.

One-shot write

$$W_0 \;=\; Z\, M_0^\dagger \tag{9b.1}$$

$$M \;=\; W_0^\dagger\, Z_\xi \tag{9b.2}$$

Equation (9b.1) computes addressing weights by projecting the episode encoding onto the reference memory. Equation (9b.2) solves the least-squares problem $\min_M \|Z_\xi - W_0 M\|_F^2$ in closed form, yielding a memory that encodes the episode in a single pass without gradient steps.

Sequential updates over many writes and forgets

$$C_i \;=\; C_{i-1} \;+\; \alpha_i\, W_i^\top W_i \tag{9b.3}$$

$$M_i \;=\; M_{i-1} \;+\; \alpha_i\, C_i^{-1}\, W_i^\top (Z_i - W_i M_{i-1}) \tag{9b.4}$$

These maintain M as the least-squares solution for the current set of episodes after arbitrary sequences of writes ($\alpha_i = +1$) and forgets ($\alpha_i = -1$). Forgetting is not a separate unlearning procedure; it is an inverse write, which aligns directly with the reconsolidation dynamics described in Appendix C.

Probabilistic read

$$\bar{W} \;=\; z\, M^\dagger, \qquad W \;\sim\; \mathcal{N}\!\left(\bar{W},\, \sigma_w^2\, I\right) \tag{9b.5}$$

$$z_r \;=\; W\, M \tag{9b.6}$$

Reading draws addressing weights from a distribution rather than selecting the maximum, which adds generative diversity to retrieval and matches the Bayesian interpretation of memory as latent-variable inference. The read-out z_r is projected and supplied to the decoder as KV-cache conditioning rather than concatenated to the input.

Capacity

Empirical results in the source show that a fixed-size memory with $K = 512$ slots recalls roughly 99 percent of stored facts at $N = K$, 82 percent at $N = 2K$, and 54 percent at $N = 4K$. Compression beyond slot count is achieved through distributed superposition: each

slot is a basis vector that participates in many memories rather than a dedicated container for one.

Larimar: Large Language Models with Episodic Memory Control. arXiv:2403.11901v4. The paper implements the Complementary Learning Systems framing with a fixed-size distributed associative memory, one-shot least-squares writes, sequential updates that compose writes and forgets, probabilistic reads, and KV-cache conditioning of the decoder. These equations specify the fast-learning side of the Section 9 loop at the level at which it can be implemented.

Engineering Note. Three things that go wrong in practice. First, if the reference memory M_0 is chosen at random it works for exact-prompt recall but degrades on rephrased queries; structured M_0 built from encodings of known facts performs markedly better on paraphrase robustness. Second, the reading noise σ_w trades diversity against perplexity; edit success stays robust up to about $\sigma_w = 0.1$ but generation quality degrades above that. Third, without a scope detector every query triggers retrieval, which destroys neighborhood specificity on out-of-scope prompts; train a small binary classifier on the encoder's embeddings to route in-scope queries through memory conditioning and out-of-scope queries through the substrate alone. The scope detector is cheap and fixes a failure mode that is otherwise hard to diagnose.

9c. Cognitive-Map Architecture: Successor Representation, TEM, Clone Cells

Section 9 describes the hippocampus-cortex loop at the level of update rules. It does not specify how the hippocampal side should be organized internally to support generalization across environments with shared structure, nor how aliased sensory states should be disambiguated, nor what the entorhinal cortex should look like computationally. This subsection adds that specification.

Variables: $T \in \mathbb{R}^{S \times S}$ is the transition matrix over S latent states. $\Psi \in \mathbb{R}^{S \times S}$ is the successor-representation matrix. $\gamma \in (0, 1)$ is the discount factor. ϕ is the path-integration encoder (MEC-like). μ is the content encoder (LEC-like). g_t is the abstract grid-like location code at time t. x_t is the sensory content at time t. p_t is the conjunctive hippocampal code at time t. v_t is the velocity or action input.

Successor representation

$$\Psi = \sum_{k=0}^{\infty} \gamma^k T^k = (I - \gamma T)^{-1} \tag{9c.1}$$

Grid cells approximate the eigenvectors of T; place cells approximate rows of Ψ. This gives the substrate a normative objective: learn a transition model whose spectral decomposition yields grid-like codes. The representation is not hand-engineered for physical space; it applies to any latent-state inference problem, including task structure, family trees, and abstract relations.

Tolman-Eichenbaum Machine split

$$g_t = \phi(g_{t-1},\, v_t) \qquad \text{(abstract path integration, reusable)} \tag{9c.2}$$

$$x_t = \mu(\text{sensory}_t) \qquad \text{(content encoding, environment-specific)} \tag{9c.3}$$

$$p_t = g_t \otimes x_t \qquad \text{(conjunctive binding)} \tag{9c.4}$$

The structural module ϕ is trained once and reused across environments. The content stream μ supplies sensory packets. The conjunctive layer binds them. Generalization emerges because a new environment with the same relational structure reuses the same ϕ and only requires new content bindings.

Clone-structured cognitive graphs

A single sensory observation is represented by multiple clones, each assigned to a distinct latent state. The system infers which clone is active given the current context, which resolves the aliasing problem in which identical sensory inputs occur in distinct latent contexts. In the codebook this means each seed packet has multiple clone entries, and retrieval is conditioned on the current path-integration state.

Vector cells with pre-credit-assigned values

Goal, border, and object vector cells carry learned values or policies attached to them, enabling zero-shot generalization to new configurations that share those relational features. Values are trained offline from behavioral statistics, which aligns with the offline replay phase of Section 9.

Representational drift as temporal remapping

Drift in hippocampal coding is not noise. Hippocampal cells conjunctively encode space, content, and time, so as time passes representations drift in a structured way that preserves temporal order within episodes. The substrate should treat drift as a feature of the temporal code rather than as something to be corrected against.

How to build a cognitive map: insights from models of the hippocampal formation. The paper synthesizes the successor-representation framework, the Tolman-Eichenbaum Machine architecture, clone-structured cognitive graphs, vector-cell value attachment, and drift-as-remapping into a unified account of hippocampal-entorhinal function. Held in Genesis Labs project knowledge.

Engineering Note. The four-factor recipe for generalization, namely factorized representations, learnable path integration, relational memory, and offline replay, is not a checklist you apply at the end. It is four constraints that have to hold jointly from the first commit. If the SNN substrate is fixed rather than learnable, the system cannot discover the abstract structural rules that make generalization possible. If the codebook has no clone mechanism, the system aliases distinct latent states with identical sensory input. If there is no offline replay mode with plasticity enabled, credit assignment is slow and structure-learning

is inefficient. If space and content are not factorized, the system fails to transfer from a trained environment to a structurally identical new one. A measurement to track: mutual information between the structural stream and the content stream. Lower is better. If it creeps up during training, the two streams are entangling and generalization will degrade.

10. Neuromodulator Broadcasts: Global Operating-Mode Signals

Arrows: DA, NE, ACh, 5-HT broadcast to all modules simultaneously.

$$\Delta w \leftarrow \Delta w \cdot (1 + d) \qquad \text{(Dopamine, plasticity and reinforcement)} \tag{17}$$

$$g \leftarrow g \cdot \mathrm{ne} \qquad \text{(Norepinephrine, arousal and gating gain)} \tag{18}$$

$$\mathrm{input}_e = \alpha_{\mathrm{ACh}} \cdot \text{bottom-up} + (1 - \alpha_{\mathrm{ACh}}) \cdot \text{top-down} \qquad \text{(Acetylcholine)} \tag{19}$$

$$\text{exploration_rate} \propto (1 + s) \qquad \text{(Serotonin, uncertainty tolerance)} \tag{20}$$

Variables: d is the dopamine reward prediction error signal. ne is the norepinephrine arousal gain. α_{ACh} is the acetylcholine modulation scalar, consistent with Section 9. s is the serotonin uncertainty tolerance scalar. Δw is the current weight update. g is the gating gain. input_e is the effective input to a cortical region, a weighted blend of bottom-up sensory drive and top-down expectation.

Timing: Every cycle. All neuromodulators broadcast in real time and update every cycle. Without explicit neuromodulation, the system has fixed hyperparameters and cannot shift between operating modes.

Yu, A.J. and Dayan, P. (2005). Uncertainty, neuromodulation, and attention. Neuron, 46(4), 681–692. DOI: 10.1016/j.neuron.2005.04.026

Avery, M.C. and Krichmar, J.L. (2017). Neuromodulatory systems and their interactions: a review of models, theories, and experiments. Frontiers in Neural Circuits, 11:108. DOI: 10.3389/fncir.2017.00108

Engineering Note. The architectural temptation is to pass d, ne, α_{ACh}, and s as explicit arguments into every forward call of every module. Resist. This creates a combinatorial mess. The cleaner pattern is a single `NeuromodulatorBus` object that modules read from during their forward pass, implemented as a dict of tensors updated once per step by a central controller. Each module that cares about a particular modulator subscribes to its key at init time. This keeps the modules themselves clean and makes it trivial to ablate individual modulators for analysis (set that key to its neutral value, which is 0 for d and s, 1 for ne, and 0.5 for α_{ACh}). The bus can be updated from biological-plausible sources (VTA for DA, LC for NE, NBM for ACh, raphe for 5-HT) or from task-driven controllers during ablation studies. Both approaches are valid; what is not valid is hardcoding modulator values as hyperparameters, which removes the entire point of the system.

Engineering Note (Receptor-Subtype Routing). Equations (17) through (20) model each neuromodulator as a single scalar. Avery and Krichmar (2017) show that the functional effect of each neuromodulator depends on which receptor subtype it binds and where that receptor is expressed. Dopamine acting on D1 receptors in superficial PFC layers blocks lateral interference and stabilizes attractor states; the same dopamine acting on D2 receptors in deep L5 neurons disinhibits thalamic input. Acetylcholine projects through two spatially distinct pathways with opposite effects: the *incremental* pathway (nucleus basalis to neocortex) enhances cue salience, while the *decremental* pathway (medial septum to hippocampus and cingulate) supports latent inhibition of irrelevant cues. These are not one scalar with two roles; they are two channels with different targets. A faithful implementation replaces each scalar with a routing table:

$$\Delta w \;\leftarrow\; \Delta w \cdot (1 + d \cdot R_{\mathrm{DA}}(\text{layer},\ \text{receptor})) \tag{17b}$$

$$\text{input}_e \;=\; \alpha^{\text{inc}}_{\text{ACh}}(\text{region}) \cdot \text{bottom-up} \;+\; (1 - \alpha^{\text{dec}}_{\text{ACh}}(\text{region})) \cdot \text{top-down} \tag{19b}$$

where R_{DA} is a fixed lookup table mapping (layer, receptor subtype) pairs to scaling factors, and the two ACh terms are separate per-region schedules rather than a single global scalar. This is cheap to implement as a small embedding indexed by module identity, and it becomes architecturally important when the substrate from Appendix D carries the heterogeneous cell classes of subsection A.a, because D1 versus D2 routing needs to know which class it is modulating. The Collins and Anastasiades synaptic-dynamics data cited in Section 5 reinforces this: dopamine arriving at L5 versus L6 PFC neurons encounters depressing versus facilitating synapses, exactly the layer-specific receptor effect this routing table is designed to capture.

10a. Multi-Scale Temporal Homeostasis (MSTH)

Multi-Scale Temporal Homeostasis: a framework for sustaining stable neural network function across catastrophic perturbations. arXiv:2602.07009v1. The source presents the first systematic implementation of coordinated four-timescale regulation at ultrafast (5 ms), fast (2 s), medium (5 min), and slow (1 to 24 hr) timescales, with a cross-scale coordinator embodying metaplasticity principles. Biological motivations cited in the source: ultra-fast synaptic depression (English et al., 2014), calcium homeostasis (Abbott and Regehr), synaptic scaling (Turrigiano), structural plasticity (Holtmaat and Svoboda, 2009), and metaplasticity (Abraham, 2008).

This subsection adds a multi-timescale homeostatic interpretation layer attachable to the existing PRAGMI architecture. The source describes coordinated regulation across ultrafast, fast, medium, and slow timescales with a cross-scale coordinator embodying metaplasticity principles. Appended as additive support for homeostatic stability.

Variables: a_t is the activity vector at time t. M_{high} and M_{low} are high-activity and low-activity regulatory masks. E_k is an emergency indicator for signal k drawn from $\{\text{mag}, \text{rate}, \text{var}, \text{mean}\}$. τ_{refract} is the refractory period preventing repeated emergency

triggers. c_t and c_{target} are the calcium level at time t and the homeostatic calcium target. ε_{Ca} is the mean absolute calcium error. A_{accum} and A_{level} are the activity accumulator and the instantaneous activity level. α_{scale} is the synaptic scaling factor. $P_{\text{accumulator}}$ and P_{recent} are the plasticity accumulator and its recent average. W_t is the weight matrix at time t. H_{system}, H_{activity}, H_{calcium}, and H_{weights} are composite and per-subsystem health scores. H_{current} and $H_{\text{stability}}$ are instantaneous and long-window health scores used to adapt the learning rate. α_{adaptive} is the cross-scale adaptive learning rate. $\rho(t)$ is a regulatory signal passed from medium to slow timescale. R_{bio} is the biological realism score, with coordinator bonus term $B_{\text{bio}}^{\text{coord}}$. The weight vector w for the realism score takes values $[3.0, 0.3, 0.4, 0.4]$, which are training artifacts and not biological quantities.

Ultra-fast: emergency detection and regulatory clamp

$$\text{Emergency}_t = \left(\sum_{k \in \{\text{mag,rate,var,mean}\}} E_k \geq 2 \right) \wedge (t - t_{\text{last}} > \tau_{\text{refract}}) \tag{10a.1}$$

$$a_t^{\text{reg}} = a_t \odot (M_{\text{high}} \cdot 0.95 + M_{\text{low}}) \tag{10a.2}$$

Fast: calcium error and correction

$$\varepsilon_{\text{Ca}} = \text{mean}(|c_t - c_{\text{target}}|) \tag{10a.3}$$

$$c_t^{\text{reg}} = c_t - 0.12 \cdot \sigma(4.0\,(c_t - c_{\text{target}})) \odot (c_t - c_{\text{target}}) \tag{10a.4}$$

Medium: activity accumulation and synaptic scaling

$$A_{\text{accum}}(t) = A_{\text{accum}}(t-1) + A_{\text{level}}(t) \tag{10a.5}$$

$$\alpha_{\text{scale}} = \begin{cases} 0.996 & \text{if recent activity too high} \\ 1.004 & \text{if recent activity too low} \end{cases} \tag{10a.6}$$

$$P_{\text{recent}} = \frac{1}{3} \sum_{j=0}^{2} P_{\text{accumulator}}[-j-1] \tag{10a.7}$$

Slow: weight decay

$$W_t^{\text{reg}} = 0.999 \cdot W_t \tag{10a.8}$$

Cross-scale: system health and adaptive learning rate

$$H_{\text{system}} = \frac{1}{3}(H_{\text{activity}} + H_{\text{calcium}} + H_{\text{weights}}) \tag{10a.9}$$

$$\alpha_{\text{adaptive}} = 0.001 \cdot H_{\text{current}} \cdot H_{\text{stability}} \tag{10a.10}$$

$$W_{\text{final}}(t) = \text{slow_regulation}(\text{medium_regulation}(W_0(t), \rho(t)), P(t)) \tag{10a.11}$$

$$R_{\text{bio}} = 1.0 - \sum_{i=1}^{4} w_i \cdot |r_i^{\text{actual}} - r_i^{\text{expected}}| + B_{\text{bio}}^{\text{coord}} \tag{10a.12}$$

$$w = [3.0,\ 0.3,\ 0.4,\ 0.4]^\top \tag{10a.13}$$

These relations naturally extend Section 10 and Appendix B by specifying an explicit hierarchy of regulatory timescales that may sit above real-time neuromodulator broadcasts and below long-horizon structural updates.

Engineering Note. The four timescales must execute in strict order within each step: ultra-fast first, then fast, then medium, then slow. Violating this order creates oscillations. Specifically, if medium synaptic scaling runs before ultra-fast emergency detection, a runaway activity spike will propagate through one full step before being clamped, which is enough to corrupt the weight scaling signal. The cleanest implementation is a single regulatory pass that runs as a hook after each forward pass, checking emergency conditions synchronously, then calcium, then scaling, then decay. Do not parallelize these across threads. The 0.95 emergency clamp, the 0.996 and 1.004 scaling factors, and the 0.999 weight decay are all multiplicative and compose, which means running them in the wrong order gives you different numerical results even though the math looks symmetric. The R_{bio} score is a diagnostic, not a loss to optimize. Logging it to TensorBoard and watching for drift over training is the right use; backpropagating through it will fight the task loss and produce nothing useful.

Summary: Implementation Priority

1. **Thalamic Gate.** Highest priority, everything else depends on it.
2. **Hippocampus feedback loop.** Resolves catastrophic forgetting.
3. **Basal Ganglia disinhibitory gating.** Replaces softmax with biological selection.
4. **Cerebellum efference copy loop.** Completes predictive coding.
5. **ACC conflict monitoring.** Connects uncertainty to attentional precision.
6. **Neuromodulator broadcasts.** Enables real-time operating mode shifts.

A. Module-to-Biology Mapping

Module	Biological Analog	Functional Role
`ThalamicRouter`	Thalamus	Multiplicative gating layer, primary input filter.
`CLS_Module`	Hippocampus / Cortex	Fast-path encoding and slow-path consolidation replay.

Module	Biological Analog	Functional Role
`StriatalGatingSelector`	Basal Ganglia	Disinhibitory selector, releases winning action from tonic bias.
`CerebellarCorrection`	Cerebellum	Parallel forward model, efference copy versus sensory actuals.
`ACC_Monitor`	Anterior Cingulate	Entropy-based conflict detection modulating $\beta_{\mathbf{NE}}$.

A.a Neural Diversity and Parallel Pathways

Appendix A maps PRAGMI modules to biological analogs at the module level. It does not specify that within each module the neurons should be heterogeneous. This subsection adds that specification. The commitment is that the SNN substrate is not a homogeneous pool of identical units; it is a structured, multi-class population with identifiable cell types, organized into parallel pathways, initialized with diversity from the start and refined by experience-dependent plasticity within class-specific bounds.

Three parallel pathways

The visual system maintains at least three partially segregated channels, namely the magnocellular, parvocellular, and koniocellular pathways, with different temporal and spatial characteristics and different cortical targets. The corresponding PRAGMI commitment is:

Magnocellular-like. Fast, transient, high temporal resolution, low spatial resolution. Candidate role: rapid context updating, novelty detection, fast operating-mode shifts.

Parvocellular-like. Slow, sustained, high spatial resolution, color-opponent in biology. Candidate role: detailed episodic content, fine-grained binding, sustained working memory.

Koniocellular-like. Small, heterogeneous, projects directly to V1 blobs in biology. Candidate role: salience, novelty, emotional tagging, direct access to the codebook without going through the standard bottom-up pipeline.

Sense versus nonsense markers

Some parameters causally determine a neuron's computational role, including time constants, thresholds, adaptation rates, the A, B, C matrices of the Appendix D state-space model. These are the sense markers. Others are correlated with function but their causal role is unestablished. These are the nonsense markers, and they remain useful as diagnostic tags that allow a population to be isolated and studied during analysis even when the underlying mechanism is not understood. Both kinds should be present in the substrate.

Compartmentalization within regions

V1 has cytochrome-oxidase-rich blobs for color processing and CO-poor interblobs for form processing. The corresponding PRAGMI commitment is that the present-moment buffer and the association cortex module should have internal compartments with different processing characteristics rather than presenting a single homogeneous surface to upstream and downstream modules.

Developmental scaffold plus experience-dependent refinement

Cell types are specified early and refined by experience within class-specific bounds. The corresponding implementation pattern is to initialize the substrate with distinct neuron classes using different parameter sets drawn from class-specific priors, then allow plasticity to fine-tune within those bounds rather than expecting undifferentiated neurons to discover specialization from scratch. This matches the staged maturation view that subsection 1a already motivates from developmental neuroscience.

Functional units are ensembles

GABAergic interneurons exist across all layers and compartments and participate in many functions. The functional unit is the dynamic assembly of multiple cell types working together, not the single-cell tuning curve. Analysis should aggregate by ensemble composition rather than by single-neuron specialization.

Neuronal chemistry and functional organization in the primate visual system. Trends in Neurosciences, 21, 344–349 (1998). DOI: 10.1016/S0166-2236(98)01262-4. The paper establishes that neurons doing different things express different molecules, that molecular markers enable population-level isolation and analysis, that at least three parallel pathways (M, P, K) coexist with different properties and cortical targets, and that developmental specification creates a scaffold which experience then refines within class bounds.

Engineering Note. The common failure mode is to instantiate every neuron in the substrate from the same parameter distribution and hope that diversity emerges from learning. It does not reliably emerge, and even when it does the resulting clusters are not interpretable because no marker identifies which unit belongs to which population. The fix is cheap: assign every unit a class label at initialization, sample its parameters from a class-specific prior, and carry the class label forward as a read-only tag used only for analysis. Three to five classes is enough to start; more can be added per module as needed. The pathway segregation is equally cheap to enforce: gate connectivity so that units of a given class preferentially connect to units of specific other classes rather than uniformly to the whole substrate. These two moves, namely class tags at init and class-aware connectivity masks, give you the M/P/K-style parallel architecture without any new learning machinery, and they make every subsequent ablation study dramatically easier to interpret.

B. Log-Dynamic Timing Hierarchy

Two timing vocabularies appear in this document. Sections 1 through 10 use functional phase labels (pre-V1, pre-relay, pre-execution, continuous, every cycle, write mode, read mode) that describe when a signal arrives relative to other signals in the same processing cycle. The oscillatory hierarchy below describes the frequency bands at which the cycles themselves occur. The two vocabularies are complementary, not alternative. A signal tagged continuous in Section 4 still ultimately rides on gamma bursts gated by theta phase as described here.

Cross-Frequency Coupling Rule: Gamma bursts are gated by the phase of the theta cycle. Feature binding only occurs when the network is in an excitable state prepared by the slower rhythms.

Regime	Frequency	Functional Role
Delta	1–4 Hz	Master scheduler: global state transitions and consolidation phases.
Theta	4–10 Hz	Phase reference: sequential trajectory encoding in MemoryCortex.
Beta	13–30 Hz	Status quo: maintains PFC goal states and suppresses irrelevant updates.
Gamma	30–100 Hz	Local binding: high-frequency bursts for cortical feature unification.

Engineering Note. If you are not using a spiking substrate, these frequencies map onto update cadences rather than literal oscillations. Delta becomes "every N global steps do consolidation," theta becomes "phase-index each sequential step within a trajectory," beta becomes "hold PFC state for K steps before allowing updates," and gamma becomes "one local binding operation per forward pass." The cross-frequency coupling rule means the gamma binding operation should be gated by a theta phase indicator: a simple sinusoidal clock with period matching the trajectory length, where binding only fires during the rising phase. This is cheap to implement (one sine eval per step, one threshold comparison) and captures the essential biology without requiring a full spiking simulator. If you are using a spiking substrate, the rates give you target firing rates for the respective populations, not simulation timesteps.

C. Reconamics Continuity Frame

Sections 1 through 10 specify the ten brain-accurate feedback loops that implement PRAGMI. They do not state the system-level commitment those loops are in service of, which is that experiential continuity must not live inside any swappable module. This appendix adds that frame. It is a prose appendix with no new equations.

The architectural commitment is that a neuromorphic stack built from PRAGMI components must preserve experience across hardware upgrades, module replacements, and sensor changes. Continuity is anchored in a persistent spiking substrate and a geometric seed-packet codebook. Reasoning modules consume reconstructed episodes but do not own them. Retrieval is treated as an intervention: reconstructed episodes can be written back, which produces reconsolidation, which is a feature when it yields compression or corrective integration and a liability when it yields drift. Drift is managed by separating the immutable identity of stored trajectories from mutable overlays such as salience weights, affect tags, and narrative bindings, so that updates are localized rather than globally rewriting content.

Five properties follow from this separation. **First, plug-compatibility**: a hardware upgrade requires modification of the driver layer and retraining of the associated projection head; it does not require discarding experiential history. **Second, independent versioning**: reasoning modules may be upgraded, replaced, or fine-tuned without altering the substrate or codebook, and the substrate may be expanded without replacing reasoning modules. **Third, cross-modal alignment as a measurable property**: projection heads are trained with neighborhood-preserving objectives so that proximity in the shared coordinate space is a principled correlate of similarity in the source modality. **Fourth, bounded reconsolidation**: drift is confined to overlays unless explicit structural consolidation is authorized, and rollback restores prior overlay states without erasing structural memory. **Fifth, auditability by construction**: because immutable and mutable components are separated, the fusion interface functions as a governance surface at which retrieval selections, reconstruction inputs, and overlay modifications are inspectable.

The world's-first framing of the Genesis stack, that hardware upgrades, module swaps, and sensor replacements cannot induce amnesia because experience never lives in the swappable parts, rests on this appendix. The Hippocratic License with Cognitive Agency Clause is the legal enforcement layer for the agency properties that this architectural separation makes real.

Reconamics: Reconstructive Memory Dynamics for Persistent AI Continuity (Genesis Labs, 2026). Companion paper: Recomics, Modular Cognitive Architecture for Embodied Systems (Genesis Labs, 2026). Both documents are held in the Genesis Labs project knowledge and together specify the continuity frame summarized above.

Engineering Note. The temptation when implementing PRAGMI is to let the reasoning module hold state, meaning to write experience into the LLM context window, into a retrieval-augmented index that lives inside the reasoning repo, or into fine-tuned weights. All three collapse the continuity frame. The test is operational: if you can delete the reasoning module and restart with a different one, and the system resumes with its accumulated experiential structure intact, the architecture is honoring Appendix C. If the reasoning module holds anything that would be lost in that operation, it is absorbing experience it should not own. The easiest way to enforce this in code is to make the substrate and codebook the only

things that get serialized between sessions, and to require reasoning modules to reconstruct their working state from the substrate on every cold start.

D. Probabilistic Spiking State-Space Substrate

The ten feedback loops in Sections 1 through 10 run on neurons. This appendix specifies what kind of neuron. The substrate is not scalar leaky integrate-and-fire. It is an n-dimensional probabilistic spiking state-space model with HiPPO initialization, population mixing, and a convolutional reformulation that enables tractable training on long sequences.

Variables: $\mathbf{h}[t] \in \mathbb{R}^n$ is the hidden state of a single neuron at time t. $\hat{A} \in \mathbb{R}^{n \times n}$ is the discretized state-transition matrix, parameterized as normal-plus-low-rank. $\hat{B} \in \mathbb{R}^{n \times 1}$ is the input-influence matrix. $C \in \mathbb{R}^{1 \times n}$ is the output-mapping matrix. $x[t]$ is the input current at time t. $p[t] \in [0, 1]$ is the Bernoulli parameter of the spike output. $s[t] \in \{0, 1\}$ is the realized spike. $K = (\hat{C}\hat{B}, \hat{C}\hat{A}\hat{B}, \ldots, \hat{C}\hat{A}^{L-1}\hat{B})$ is the convolution kernel of length L. W_{mix} is the SpikeMixer population-aggregation weight matrix.

Per-neuron recurrence

$$\mathbf{h}[t] \;=\; \hat{A}\,\mathbf{h}[t-1] \;+\; \hat{B}\,x[t] \tag{D1}$$

$$p[t] \;=\; \sigma(C\,\mathbf{h}[t]) \tag{D2}$$

$$s[t] \;\sim\; \text{Bernoulli}(p[t]) \tag{D3}$$

Convolutional reformulation for parallel training

$$p[i] \;=\; \sigma((K * x)_i), \qquad K_j \;=\; \hat{C}\,\hat{A}^{\,j}\,\hat{B} \tag{D4}$$

Population mixing

$$f_{\text{mix}}[t] \;=\; \text{GELU}(x[t]\,W_{\text{mix}}) \tag{D5}$$

HiPPO initialization

The matrix A is initialized with HiPPO-LegS coefficients, giving each neuron a built-in polynomial approximation basis for compressing its input history. The NPLR decomposition of A enables the kernel K to be computed in $\mathcal{O}(L)$ time via Cauchy-kernel machinery, removing the BPTT bottleneck that makes scalar-LIF substrates computationally prohibitive on long experience streams.

Sparsity

Under the probabilistic spike formulation, roughly 90 percent of the population is dormant at any given step as an emergent property of training rather than an imposed constraint. This

matches the PRAGMI design intent that most moments should not be committed to memory, and it aligns with the sparsity requirement on the hippocampal representation H in Section 9.

P-SpikeSSM: Harnessing Probabilistic Spiking State Space Models for Long-Range Dependency Tasks. ICLR 2025 (arXiv preprint). The paper establishes that the scalar membrane potential of LIF neurons is a fundamental representational ceiling, and replaces it with an n-dimensional state that admits parallel training via convolution, HiPPO-based polynomial history compression, probabilistic spike generation, and population mixing through the SpikeMixer layer.

Engineering Note. The substrate equations above are not optional. Every equation in Sections 1 through 10 implicitly assumes something about what the variables live inside, and scalar LIF cannot carry the rich history that Section 9 demands or the long-range dependencies that the PFC goal vector in Section 5 requires. Three implementation traps: first, do not use batch normalization in the ClampFuse layer of an online streaming setup; replace it with layer normalization or EMA-based statistics or the normalization statistics will drift as the stream advances. Second, do not treat the convolution formulation as fixed-length; PRAGMI's experience stream is continuous, so use a sliding window or an IIR approximation rather than a hard-coded L. Third, serialize the full hidden-state vector $\mathbf{h}[t]$, not just a scalar membrane potential, when saving a session; the whole point of the upgrade is that the polynomial projection carries history across the session boundary.